

Beginner JavaScript Lecture Notes

Setup & Prerequisites: Environment Configuration

Before writing JavaScript code, we need an environment to run it. We will use Node.js to execute JavaScript on our computers and Visual Studio Code (VS Code) as our code editor.

1. How to Download and Install Node.js

Node.js is a runtime environment that allows you to run JavaScript outside of a web browser.

- For Windows: Go to the official Node.js website (nodejs.org). Download the "LTS" (Long Term Support) version Windows Installer (.msi). Double-click the downloaded file and follow the installation wizard, leaving all settings at their default.
- For Mac: Go to nodejs.org and download the macOS Installer (.pkg). Double-click the file and follow the installation prompts. Alternatively, if you use Homebrew, you can install it via your terminal by typing: `brew install node`

To verify the installation, open your Terminal (Mac) or Command Prompt (Windows) and type `node -v`. It should print the installed version number.

2. How to Install Code Runner on VS Code

Code Runner is an extension that allows you to execute code with a single click.

1. Open Visual Studio Code.
2. Click on the Extensions icon on the left sidebar (it looks like four squares).
3. In the search bar, type "Code Runner".
4. Look for the extension provided by "Jun Han" and click Install.
5. Once installed, you will see a small "Play" button (▶) in the top-right corner of your code editor. Clicking this will run your JavaScript file.

Module 1: JavaScript Fundamentals

1. JavaScript Introduction

Meaning: JavaScript (JS) is a high-level, versatile programming language primarily used to make web pages interactive. Today, it is also widely used for building backend servers, mobile apps, and more.

Real-Life Example: Think of building a house. HTML is the structure (bricks and walls), CSS is the styling (paint and decorations), and JavaScript is the electricity and plumbing that makes the house functional and interactive.

2. Variables

Meaning: A variable is a container used to store data values. In modern JavaScript, we declare variables using `let` and `const`.

Real-Life Example: Imagine a labeled storage box. The label on the box is the variable name, and the items inside are the data.

Code:

```
// Using let (value can change)
let age = 20;
age = 21; // This is perfectly fine

// Using const (value cannot change)
const birthYear = 2005;
// birthYear = 2006; // ERROR! You cannot reassign a const
```

3. Data Types

Meaning: Data types define the kind of data a variable can hold.

Code:

```
let studentName = "Alice"; // String
let score = 95.5;           // Number
let isPassed = true;        // Boolean
let futureGoal;             // Undefined
let currentJob = null;      // Null
```

4. Operators

Meaning: Symbols used to perform operations on variables and values.

Code:

```
let a = 10;
let b = 5;
console.log(a + b); // 15
console.log(a > b); // true (Comparison)
console.log(a === 10 && b === 5); // true (Logical AND)
```

5. Type Casting (Type Conversion)

Meaning: Converting data from one type to another. It can be Implicit (done automatically by JS) or Explicit (done manually by the programmer).

Code:

```
// Implicit (Coercion)
let result = "5" + 2;
console.log(result); // Outputs "52" (Number 2 is coerced into a String)
```

```
// Explicit
let numStr = "100";
let actualNumber = Number(numStr); // Manually converted to Number
```

Module 2: Control Flow & Scope

1. Conditions

Meaning: Conditional statements are used to perform different actions based on different conditions. The main keywords are if, else if, and else.

Real-Life Example: A traffic light. IF the light is red, you stop. ELSE IF the light is yellow, you slow down. ELSE (if green), you go.

Code:

```
let trafficLight = "red";
if (trafficLight === "red") {
  console.log("Stop!");
} else {
  console.log("Go!");
}
```

2. Loops

Meaning: Loops can execute a block of code a number of times as long as a specified condition is true.

Code:

```
for (let i = 1; i <= 5; i++) {
  console.log("Lap number: " + i);
}
```

3. Scope

Meaning: Scope determines the accessibility (visibility) of variables. If a variable is out of scope, it cannot be used.

Code:

```
let globalVar = "I am everywhere!"; // Global scope
if (true) {
  let blockVar = "I am trapped inside this block!"; // Block scope
}
```

4. Basic Problem-Solving in JavaScript

Problem-solving involves breaking a complex task into smaller steps using logic, loops, and conditions. Always map out your logic before typing code.

Module 3: Data Structures (Strings, Arrays & Objects)

1. Strings

Meaning: Strings are sequences of characters used to represent text. They are immutable (cannot be changed once created) and use zero-based indexing.

Code:

```
let msg = "Hello World";
console.log(msg.length); // 11
console.log(msg.toUpperCase()); // "HELLO WORLD"
console.log(msg.slice(0, 5)); // "Hello"
```

2. Arrays

Meaning: An array is a special variable that can hold more than one value at a time. It's an ordered collection of data.

Code:

```
let fruits = ["Apple", "Banana", "Mango"];
fruits.push("Orange"); // Adds to the end
fruits.pop(); // Removes the last element
console.log(fruits[0]); // "Apple"
```

3. Objects

Meaning: Objects are collections of key-value pairs. Keys are strings, and values can be any data type (including other objects or arrays).

Code:

```
let user = {
  name: "Alice",
  age: 25,
  isStudent: true
};
console.log(user.name); // Dot notation
console.log(user["age"]); // Bracket notation
```

Module 4: Functions, ES6 & Advanced Concepts

1. Functions

Meaning: A JavaScript function is a block of code designed to perform a particular task. It is executed when "invoked" (called).

Code:

```
function greet(name) {
  return "Hello, " + name + "!";
}
```

```
}  
console.log(greet("Bob"));
```

2. Higher-Order Functions

Meaning: A function that takes another function as an argument, or returns a function as its result.

3. Lambda (Arrow) Functions

Meaning: Arrow functions provide a shorter syntax for writing functions. They are the standard in modern JS development.

Code:

```
const addArrow = (a, b) => a + b;
```

4. Hoisting

Meaning: Hoisting is JavaScript's default behavior of moving variable and function declarations to the top of their containing scope before code execution.

5. Closures

Meaning: A closure is a function that remembers its outer variables and can access them even after the outer function has finished executing.

Code:

```
function createGreeting(greetingWord) {  
  return function(name) {  
    console.log(greetingWord + ", " + name);  
  };  
}  
const sayHi = createGreeting("Hi");  
sayHi("Alice"); // Outputs: Hi, Alice
```

6. Array Iteration Methods (Map, Filter, Reduce)

Meaning: Built-in higher-order functions used to iterate over arrays and manipulate data without traditional for-loops.

Code (Map - transforms data evenly):

```
let numbers = [1, 2, 3];  
let doubled = numbers.map(num => num * 2);  
console.log(doubled); // [2, 4, 6]
```

Code (Filter - extracts specific items):

```
let scores = [45, 80, 92];  
let passed = scores.filter(score => score >= 50);  
console.log(passed); // [80, 92]
```

Code (Reduce - aggregates data to a single value):

```
let prices = [10, 20, 30];  
let total = prices.reduce((acc, curr) => acc + curr, 0);  
console.log(total); // 60
```

7. ES6 Basics

Meaning: ECMAScript 2015 (ES6) introduced major new features to JavaScript, making code cleaner and more efficient.

Code:

```
// Template Literals  
let name = "Bob";  
console.log(`Welcome back, ${name}!`);
```

```
// Destructuring  
let user = { id: 1, role: "admin" };  
let { id, role } = user;
```

```
// Spread Operator  
let arr1 = [1, 2];  
let arr2 = [...arr1, 3, 4]; // [1, 2, 3, 4]
```

8. Object Manipulation

Meaning: Advanced techniques to interact with and transform objects dynamically using `Object.keys()`, `Object.values()`, and `Object.entries()`.

Code:

```
let car = { make: "Toyota", model: "Corolla" };  
console.log(Object.keys(car)); // ["make", "model"]
```

Module 5: Asynchronous JavaScript

1. Synchronous vs. Asynchronous JavaScript

Meaning:

- **Synchronous:** Code is executed line by line, from top to bottom. If one line of code takes a long time (like a huge calculation), it "blocks" the rest of the code from running until it finishes. This is JavaScript's default behavior (it is single-threaded).
- **Asynchronous:** Code allows the program to start a long-running task (like fetching data from the internet) and continue executing other code while waiting for that task to finish. It prevents the browser from freezing.

Real-Life Example:

- Sync: Waiting in a long line at a coffee shop. You cannot do anything else until the person in front of you gets their coffee.
- Async: Ordering food at a restaurant. You place your order, sit down and talk with friends (main thread continues), and eventually, the waiter brings your food (async task finishes).

2. Timers: `setTimeout` and `setInterval`

Meaning: These are built-in browser functions used to schedule code execution in the future.

- `setTimeout`: Executes a block of code exactly ONCE after a specified delay (in milliseconds).
- `setInterval`: Executes a block of code REPEATEDLY at a specified interval (in milliseconds) until it is manually stopped using `clearInterval`.

Code Example:

```
// setTimeout Example (Runs once after 2 seconds)
console.log("Start");
setTimeout(() => {
  console.log("This runs after 2000 milliseconds");
}, 2000);
console.log("End"); // Notice that "End" will print BEFORE the timeout message!
```

```
// setInterval Example (Runs every 1 second)
let count = 0;
const timerId = setInterval(() => {
  count++;
  console.log(`Tick ${count}`);
  if (count === 3) {
    clearInterval(timerId); // Stops the interval after 3 ticks
    console.log("Timer stopped!");
  }
}, 1000);
```

3. Callbacks & Sequential Execution

Meaning: A callback is a function passed as an argument to another function, which is executed later once an asynchronous operation completes.

Important Points: Before Promises and Async/Await, callbacks were the main way to force asynchronous tasks to run sequentially (one after another). However, deeply nested callbacks lead to "Callback Hell" (code that looks like a pyramid and is hard to read).

Code Example (Executing tasks sequentially):

```
function stepOne(callback) {
```

```

    setTimeout(() => {
      console.log("Step 1 completed");
      callback();
    }, 1000);
  }

function stepTwo(callback) {
  setTimeout(() => {
    console.log("Step 2 completed");
    callback();
  }, 1000);
}

function stepThree() {
  setTimeout(() => {
    console.log("Step 3 completed. All done!");
  }, 1000);
}

// To make them run in order, we nest the callbacks:
stepOne(() => {
  stepTwo(() => {
    stepThree();
  });
});

```

4. Promises

Meaning: A Promise is an object representing the eventual completion (or failure) of an asynchronous operation. It is a cleaner alternative to nested callbacks.

Code Example:

```

const checkWeather = new Promise((resolve, reject) => {
  let isSunny = true;
  setTimeout(() => {
    if (isSunny) {
      resolve("It's a beautiful day!");
    } else {
      reject("It's raining.");
    }
  }, 1000);
});

checkWeather

```



```
.then((message) => console.log(message)) // Runs if resolved
.catch((error) => console.log(error)); // Runs if rejected
```

5. Async / Await

Meaning: Introduced in ES8, async/await allows you to write asynchronous code that looks and behaves like synchronous code, making it much easier to read.

Code Example:

```
// Using async/await
async function displayUser() {
  try {
    console.log("Fetching user...");
    let user = await getUserData(); // Code pauses here until data arrives
    console.log("User found:", user.name);
  } catch (error) {
    console.log("Error fetching user");
  }
}
// displayUser();
```

Assignment 4: Asynchronous JavaScript

Question 1 (Promises): Create a Promise named 'bakeCake' that resolves with the string 'Cake is ready!' after 3 seconds. Consume the promise using .then() and print the result.

```
// Answer to Q1
const bakeCake = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Cake is ready!");
  }, 3000);
});

bakeCake.then((message) => {
  console.log(message);
});
```

Question 2 (Async/Await): Write an async function called 'processOrder'. Inside it, await the 'bakeCake' promise from Question 1, and console.log the result. Wrap your code in a try/catch block.

```
// Answer to Q2
async function processOrder() {
  try {
    console.log("Starting order...");
    const result = await bakeCake;
    console.log(result);
  }
}
```

```
        console.log("Order complete!");
    } catch (error) {
        console.error("Baking failed:", error);
    }
}

// processOrder();
```